

Project Report

COMP2021 Object-Oriented Programming (Fall 2025)

Group 18

Members and contribution percentages:

Meng Zihan: 25%

Huang Junhao: 25%

Luo Yeyushan: 25%

He Liuying: 25%

1. Introduction

This report outlines the architectural design and technical implementation of Clevis (Command Line Vector Graphics Software). Completed by Group 18 for the COMP2021 Object-Oriented Programming course at PolyU, this project demonstrates the practical application of core OOP principles—Encapsulation, Inheritance, Polymorphism, and Abstraction. The software provides a robust command-line interface (CLI) for users to create, manipulate, and organize vector shapes, backed by comprehensive error handling and state management systems.

2. The Command Line Vector Graphics Software (Clevis)

In this section, we describe first the overall design of the Clevis tool and then the implementation details of the requirements.

2.1 Design

This section provides details of the architectural decisions and the rationale behind our object-oriented design choices.

2.1.1 System Architecture (MVC)

The system follows a modular architecture inspired by the Model-View-Controller (MVC) pattern to ensure high cohesion, low coupling, and scalability:

Model Layer (Logic & Data): Consists of the “ShapeManager” and all shape classes (Line, Circle, Rectangle, Square, Group). This layer handles the core business logic, data management, and mathematical calculations.

View Layer (Presentation): Represented by the “CommandLogger”. It records all operations into HTML and TXT files, providing a persistent visual record of the user's session.

Controller Layer (Coordination): Includes the “Commander” for parsing user input and the Clevis class for coordinating the overall workflow and dispatching commands to the Model.

2.1.2 Application of OOP Principles

Beyond the basic structure, we applied four key OOP principles to improve code quality:

Abstraction:

We hid the complex mathematical logic behind simple method calls. Users invoking commands like `move`, `getBoundingBox`, or `containsPoint` do not need to understand the underlying geometry (e.g., vector projection for lines or Euclidean distance for circles). These details are abstracted away within the specific shape classes.

Encapsulation:

We enforced strict access control to protect the internal state of objects. Attributes such as coordinates and dimensions are private or protected, accessible only through controlled interfaces. This prevents invalid state modifications and ensures proper interaction between objects. This principle was also applied in the Controller layer to handle user input complexity. The helper methods, `tryParseDouble()` and `isValidName()`, encapsulate the detailed logic for type checking and illegal character validation, shielding the Model layer (ShapeManager) from potential input errors.

Inheritance (Design Choice: Abstract Class vs. Interface):

We implemented Shape as an abstract class rather than an interface. This was a strategic decision to improve Code Reusability.

Since all shapes share common attributes—specifically a unique name and a `zWeight` for layering—using an abstract class allowed us to define these fields and their standard getters once in the parent class.

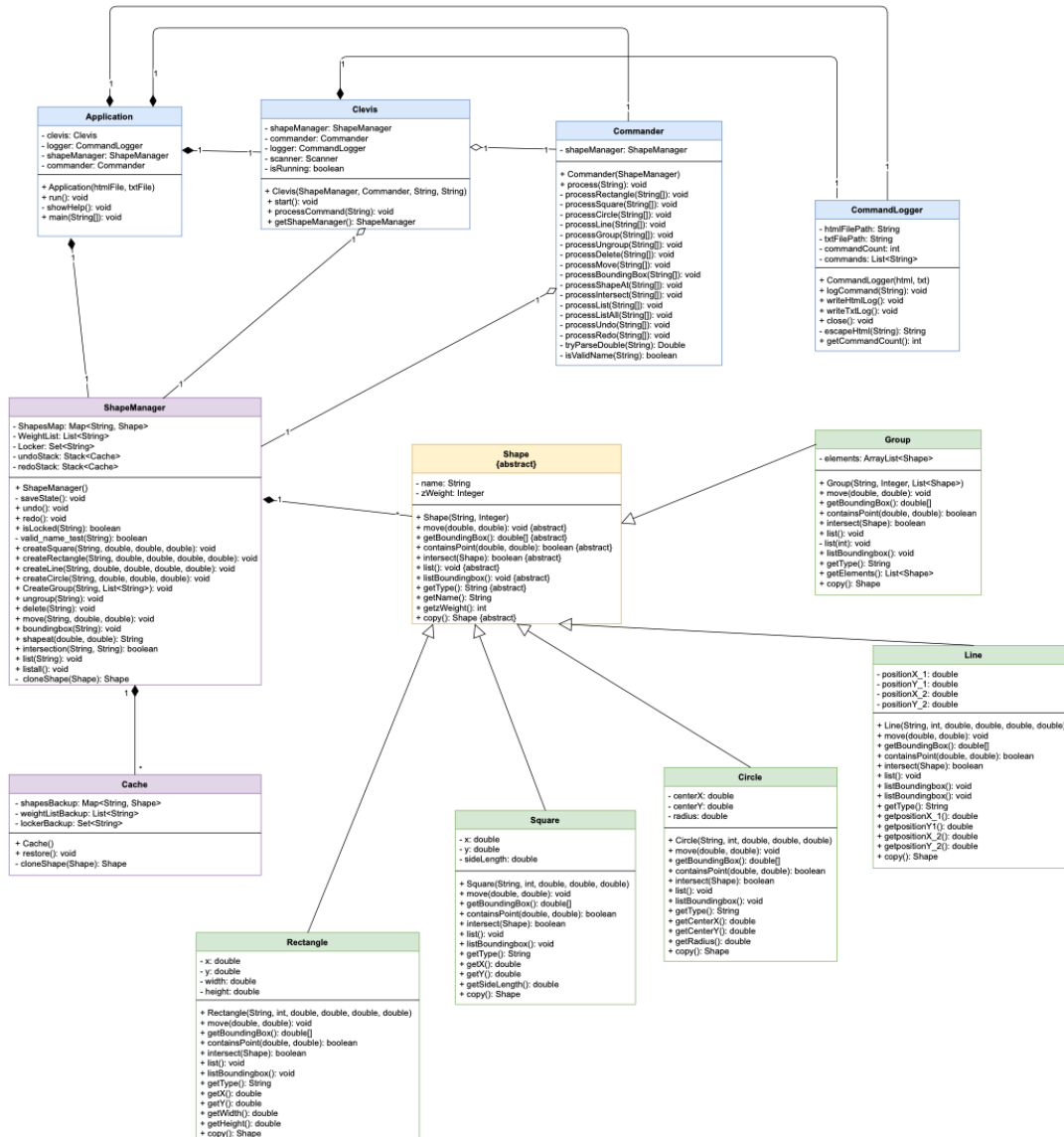
Had we used an interface, we would have violated the DRY (Don't Repeat Yourself) principle by duplicating these field definitions in every single subclass.

Polymorphism:

The system interacts with all graphical elements via the generic `Shape` reference. This decoupling allows the `ShapeManager` to manage diverse objects (lines, groups, circles) uniformly.

2.1.3 Scalability and Extensibility

The adherence to these principles significantly enhances the project's scalability. For instance, adding a new shape (e.g., `Triangle`) or a new command does not require altering the existing logic of the Model layer. Developers simply need to extend the `Shape` class and implement the required abstract methods, leaving the core architecture untouched. This "Open/Closed" nature ensures the system remains robust as it grows.



If the diagram is not clear, please check the folder file named UML_ClassDiagram.PDF

2.1.4 Debugging and Refactoring: Enhancing Input

During the integration testing phase, we identified a critical oversight in our initial implementation: the lack of robust input validation in the Controller layer. Initially, the system assumed that user inputs would always adhere to the expected format. Consequently, entering non-numeric characters for coordinates (e.g., rectangle r1 10 abc 20 20) or providing empty commands caused the application to crash immediately with a `NumberFormatException` or `NullPointerException`.

Modification and Fix: To address this, we refactored the `Commander.java` class (Controller Layer). We introduced defensive programming techniques to validate inputs before passing them to the `ShapeManager`:

Safe Parsing: We implemented a helper method, `tryParseDouble()`, which attempts to parse string arguments into double values. If parsing fails, it catches the exception and prints a user-friendly error message instead of terminating the program.

Name Validation: We added an `isValidName()` method using Regular Expressions (Regex) to ensure that shape names contain only valid alphanumeric characters, preventing logical errors in data storage.

Null Checks: We added checks at the beginning of the `process()` method to handle null or empty input strings gracefully.

This debugging process significantly improved the stability of the Clevis CLI, ensuring that the application remains responsive even when encountering invalid user inputs.

2.2 Implementation of Requirements

[REQ1]

- 1) The requirement is implemented.
- 2) **Implementation:** We utilized a dedicated “`CommandLogger`” class. This class initializes file writers for both HTML and TXT formats using paths provided via command-line arguments. It captures every valid command executed during the session and appends it to the logs in real-time.
- 3) **Error Handling:** The `Commander` first uses the `isValidName()` mechanism to validate that the provided log file paths/names contain only legal characters. The implementation then uses try-with-resources blocks to ensure file streams are closed, catching `IOException` if paths are unwritable.

[REQ2]

- 1) The requirement is implemented.
- 2) **Implementation:** We defined a “`Rectangle`” class extending the abstract “`Shape`” class. It stores the top-left coordinate (x, y), width, and height.
- 3) **Error Handling:** The `Commander` first uses the `isValidName()` mechanism to validate the shape name. It then validates all four parameters (x, y, w, h) as

valid double types. Subsequently, the ShapeManager validates that dimensions are positive and ensures the shape name is unique.

[REQ3]

- 1) The requirement is implemented.
- 2) Implementation: We defined a “Line” class extending “Shape”. It encapsulates a vector between two coordinate points: (positionX_1, positionY_1) and (positionX_2, positionY_2).
- 3) Error Handling: The Commander first uses the isValidName() mechanism to validate the shape name. It then validates all coordinate parameters (x1, y1, x2, y2) as valid double types. The ShapeManager then verifies the uniqueness of the name.

[REQ4]

- 1) The requirement is implemented.
- 2) Implementation: We defined a “Circle” class extending “Shape”. It is defined by a center point (centerX, centerY) and a radius.
- 3) Error Handling: The Commander first uses the isValidName() mechanism to validate the shape name. It then validates all parameters (x, y, r) as valid double types. Subsequently, the ShapeManager ensures the radius is a positive floating-point number and checks for duplicate names.

[REQ5]

- 1) The requirement is implemented.
- 2) Implementation: We defined a “Square” class extending “Shape”. It is defined by a top-left coordinate (x, y) and a single (sideLength) parameter.
Design Note: We chose not to have “Square” inherit from “Rectangle” to avoid parameter redundancy and Liskov Substitution Principle violations.
- 3) Error Handling: The Commander first uses the isValidName() mechanism to validate the shape name. It then validates all parameters (x, y, l) as valid double types. The ShapeManager then rejects negative side lengths and duplicate shape names.

[REQ6]

- 1) The requirement is implemented.
- 2) Implementation: We utilized the Composite Design Pattern. The “Group” class extends “Shape” and contains a list of other Shape objects. Operations called on a “Group” are delegated recursively to its children.
- 3) Error Handling: The Commander first uses the isValidName() mechanism to validate the new group name (n) and all component names (n1, n2, ...). The system then validates that all target shapes exist and are not already "locked" within another group. It also prevents the creation of empty groups.

[REQ7]

- 1) The requirement is implemented.
- 2) Implementation: The ShapeManager.ungroup() method retrieves the target group, extracts its children, and recursively "unlocks" them (removes them from the Locker set). Most significantly, it restores the shapes to the Z-order list to ensure correct layering.
- 3) Error Handling: Checks if the target exists and is actually an instance of “Group”.

[REQ8]

- 1) The requirement is implemented.
- 2) Implementation: The ShapeManager.delete() method removes references to the target shape from the primary ShapesMap, the Z-order WeightList, and the Locker.
- 3) Error Handling: Prevents deletion of shapes that do not exist or are currently locked inside a group (requiring ungrouping first).

[REQ9]

- 1) The requirement is implemented.
- 2) Implementation: Every subclass of “Shape” implements an abstract getBoundingBox() method returning [x, y, width, height].
Atomic Shapes: Calculated via geometry.
Group: Calculates the union (min X/Y and max X/Y) of all member bounding boxes.
- 3) Error Handling: Handles formatting of output to two decimal places.

[REQ10]

- 1) The requirement is implemented.
- 2) Implementation: A move(dx, dy) method is polymorphic. Atomic shapes update their coordinates by adding the delta. Group objects propagate the move command recursively to all children.
- 3) Error Handling: The Commander layer first validates the displacement parameters (dx, dy) as valid double types. The ShapeManager then prevents moving locked shapes individually.

[REQ11]

- 1) The requirement is implemented.
- 2) Implementation: All shapes implement containsPoint(px, py) with a standard tolerance of 0.05 units. The selection logic traverses the WeightList in reverse order (from newest/top-most to oldest/bottom-most) to select the visually "top" shape.
- 3) Error Handling: The Commander layer first validates the query point coordinates (x, y) as valid double types. Additionally, the system manages empty canvases and handles invalid shape names.

[REQ12]

- 1) The requirement is implemented.
- 2) Implementation: Implemented via Axis-Aligned Bounding Box (AABB) collision detection. The intersect (Shape other) method compares the getBoundingBox() results of two shapes to check for overlap.
- 3) Error Handling: Checks for self-intersection and validity of target shapes.

[REQ13]

- 1) The requirement is implemented.
- 2) Implementation: The list() method is overridden in each class to print specific attributes. Groups recursively list their children.
- 3) Error Handling: Returns error messages if the requested shape name is not found in the registry.

[REQ14]

- 1) The requirement is implemented.

- 2) Implementation: The ShapeManager.listAll() method iterates through the WeightList in reverse order (rendering order from top to bottom) and invokes the polymorphic list() method for each entry.
- 3) Error Handling: Detects and reports an empty canvas state.

[REQ15]

- 1) The requirement is implemented.
- 2) Implementation: The “quit” command triggers a shutdown in Application.java. It closes the “Scanner” input stream, finalizes the CommandLogger files, and terminates the JVM.
- 3) Error Handling: Ensures file streams are flushed before exit to prevent data loss.

[BON1]

- 1) This requirement is NOT implemented.

[BON2]

- 1) This requirement is implemented.
- 2) Implementation: Implemented using the Memento Design Pattern combined with the Prototype Design Pattern. We maintain two stacks: undoStack and redoStack. Deep Copying Strategy: Instead of a centralized cloning utility using instanceof checks, we delegated the cloning responsibility to the shapes themselves. The “ShapeManager” simply iterates through the current shapes and calls the polymorphic cloneShape() method on each. This ensures that complex structures, especially recursive “Group” hierarchies, are cloned correctly and cleanly.
- 3) Error Handling: Handles boundary conditions, such as attempting to undo/redo when the respective stacks are empty.

3. Learning-to-Learn

3.1 Reflection on Experiences

This project served as a vital bridge, allowing us to translate theoretical computer science concepts into tangible software engineering reality. We moved beyond simply studying lecture materials to actively applying core object-oriented programming principles—such as encapsulation, inheritance, and polymorphism—in a functional application.

The implementation of basic features proceeded smoothly, a direct result of our solid foundational understanding and effective teamwork. However, the bonus tasks, particularly the Undo/Redo function, presented a significant challenge that extended beyond the scope of classroom teaching. This required substantial self-learning and problem-solving; for instance, implementing the Memento pattern demanded a deep understanding of Java references versus deep copies to prevent state corruption.

This process was further refined when we realized the need to validate string content (using `isValidName()`) in addition to numerical type, solidifying our understanding that comprehensive input validation must cover both type and character set integrity.

This process provided us with practical lessons in identifying core runtime vulnerabilities and applying the principles of encapsulation and centralized error handling to build a more resilient and user-friendly system, directly fulfilling the requirement for robust error management.

We learned that technical prowess must be paired with strong communication. Solving complex problems, like the Z-order logic for ungrouping operations, involved extensive whiteboarding and debate. This collaborative process ultimately yielded a more robust and well-considered solution than any individual could have produced alone.

Through this experience, we gained a profound appreciation for the gap between theory and practice and the critical role of teamwork in tackling difficult technical problems. Although we did not complete all the bonus features, we managed to implement several through determined collaborative effort. This process provided us with invaluable experience in learning under pressure and resolving complex issues, underscoring the dynamic nature of real-world software development.

3.2 Plan for Improvement

Based on our project experience, we have formalized a structured self-learning methodology centered on a "Divide, Conquer, and Integrate" approach. This strategy involves breaking down complex tasks into manageable, atomic units—such as "file parsing," "object serialization," and "error handling"—enabling team members to focus on specific functional components with clear responsibility boundaries.

Moving forward, we will adopt a test-driven mindset, emphasizing incremental verification of individual components before integration, rather than relying solely on end-stage testing. Furthermore, we plan to cultivate a consistent code review culture, where regular peer evaluations will not only help catch logic errors early but also deepen collective understanding through explanation and discussion.

This segmented development approach allowed us to acquire new knowledge from scratch and work efficiently, while the final integration phase demonstrated how well our individual contributions complemented each other. We have learned that significant achievements are built through accumulated small steps; by dividing large tasks into visible and measurable units, we can navigate complex challenges more systematically and ensure consistent progress in our future learning and projects.